Official Objective Checklist

Generated from the Markdown study system. Labs with executable code remain in the `labs/` folder. This is study material, not a Cisco exam dump and not a pass guarantee.

CCNA Automation 200-901 v1.1 — Objective Checklist

Primary authority: `200-901-CCNAAUTO_v.1.1.pdf`.
Study text: `CCNAAUTO_COMPLETE_STUDY_GUIDE.md` (same content also split under `study-guide/`).

Mark a row complete only when tracker Status matches the verb (construct/interpret/utilize/troubleshoot → **Can Perform** or **Can Interpret**; describe/compare/explain → **Understand**+).

Obj	Official objective	Guide section	Lab	Status
1.0	Software Development and Design — **15%**	Domain 1 intro	—	
1.1	Compare data formats (XML, JSON, and YAML)	1.1	`labs/02_data_formats`	
1.2	Describe parsing of common data format (XML, JSON, and YAML) to Python data structures	1.2	`parse_formats.py`	
1.3	Describe the concepts of test-driven development	1.3	`test_subnet.py`	
1.4	Compare software development methods (agile, lean, and waterfall)	1.4	—	
1.5	Explain the benefits of organizing code into methods / functions, classes, and modules	1.5	`functions_classes_modules.py`	
1.6	Explain the advantages of common design patterns (MVC and Observer)	1.6	—	
1.7	Explain the advantages of version control	1.7	`labs/04_git`	
1.8	Utilize common version control operations with Git	1.8	Git lab in LAB_SETUP	
1.8.a	Clone	1.8.a	Git lab	
1.8.b	Add/remove	1.8.b	Git lab	
1.8.c	Commit	1.8.c	Git lab	
1.8.d	Push / pull	1.8.d	Git lab	
1.8.e	Branch	1.8.e	Git lab	
1.8.f	Merge and handling conflicts	1.8.f	Git lab	
1.8.g	diff	1.8.g	`example.diff`	
2.0	Understanding and Using APIs — **20%**	Domain 2 intro	—	
2.1	Construct a REST API request to accomplish a task given API	2.1	Postman + `rest_client.py`	

Obj	Official objective	Guide section	Lab	Status
	documentation			
2.2	Describe common usage patterns related to webhooks	2.2	Webex events (optional)	
2.3	Describe the constraints when consuming APIs	2.3	Rate-limit discussion in `troubleshoot_http.py`	
2.4	Explain common HTTP response codes associated with REST APIs	2.4	`rest_client.py` status lab	
2.5	Troubleshoot a problem given the HTTP response code, request and API documentation	2.5	`troubleshoot_http.py`	
2.6	Interpret the parts of an HTTP response (response code, headers, body)	2.6	Postman	
2.7	Utilize common API authentication mechanisms: basic, custom token, and API keys	2.7	`rest_client.py`	
2.8	Compare common API styles (REST, RPC, synchronous, and asynchronous)	2.8	—	
2.9	Construct a Python script that calls a REST API using the requests library	2.9	`rest_client.py`	
3.0	Cisco Platforms and Development — **15%**	Domain 3 intro	—	
3.1	Construct a Python script that uses a Cisco SDK given SDK documentation	3.1	`sdk_pattern.py`	
3.2	Describe the capabilities of Cisco network management platforms and APIs (Meraki, Cisco Catalyst Center, ACI, Cisco Catalyst SD-WAN, and NSO)	3.2	DevNet docs + sandbox	
3.3	Describe the capabilities of Cisco compute management platforms and APIs (UCS Manager and Intersight)	3.3	DevNet docs	
3.4	Describe the capabilities of Cisco collaboration platforms and APIs (Webex, Webex devices, Cisco Unified	3.4	`webex_rooms.py`	

Obj	Official objective	Guide section	Lab	Status
	Communications Manager including AXL and UDS interfaces)			
3.5	Describe the capabilities of Cisco security platforms and APIs (XDR, Firepower, Secure Connect, Secure Endpoint, ISE, and Secure Malware Analytics)	3.5	DevNet security docs	
3.6	Describe the device level APIs and dynamic interfaces for IOS XE and NX-OS	3.6	IOS XE sandbox	
3.7	Describe the appropriate DevNet resource for a given scenario (Sandbox, Code Exchange, support, forums, Learning Labs, and API documentation)	3.7	developer.cisco.com	
3.8	Apply concepts of model driven programmability (YANG, RESTCONF, and NETCONF) in a Cisco environment	3.8	`labs/06_yang_netconf_restconf`	
3.9	Construct code to perform a specific operation based on a set of requirements and given API reference documentation	3.9	`labs/05_cisco_apis`	
3.9.a	Obtain a list of network devices by using Meraki, Cisco Catalyst Center, ACI, Cisco Catalyst SD-WAN, or NSO	3.9.a	`meraki_list_devices.py`, `catalyst_center_devices.py`	
3.9.b	Manage spaces, participants, and messages in Webex	3.9.b	`webex_rooms.py`	
3.9.c	Obtain a list of clients / hosts seen on a network using Meraki or Cisco Catalyst Center	3.9.c	Meraki clients call	
4.0	Application Deployment and Security — **15%**	Domain 4 intro	—	
4.1	Describe the benefits of edge computing	4.1	—	
4.2	Describe the attributes of different application deployment models (private cloud, public cloud, hybrid cloud, and edge)	4.2	—	

Obj	Official objective	Guide section	Lab	Status
4.3	Describe the attributes of these application deployment types	4.3	—	
4.3.a	Virtual machines	4.3.a	—	
4.3.b	Bare metal	4.3.b	—	
4.3.c	Containers	4.3.c	Docker lab	
4.4	Describe components for a CI/CD pipeline in application deployments	4.4	—	
4.5	Construct a Python unit test	4.5	`test_subnet.py`	
4.6	Interpret contents of a Dockerfile	4.6	`labs/07_docker/Dockerfile`	
4.7	Utilize Docker images in local developer environment	4.7	`docker build` / `run`	
4.8	Describe application security issues related to secret protection, encryption (storage and transport), and data handling	4.8	`labs/.env`	
4.9	Explain how firewall, DNS, load balancers, and reverse proxy in application deployment	4.9	—	
4.10	Describe top OWASP threats (such as XSS, SQL injections, and CSRF)	4.10	—	
4.11	Utilize Bash commands (file management, directory navigation, and environmental variables)	4.11	WSL Ubuntu	
4.12	Describe the principles of DevOps practices	4.12	—	
5.0	Infrastructure and Automation — **20%**	Domain 5 intro	—	
5.1	Describe the value of model driven programmability for infrastructure automation	5.1	YANG lab	
5.2	Compare controller-level to device-level management	5.2	—	

Obj	Official objective	Guide section	Lab	Status
5.3	Describe the use and roles of network simulation and test tools (such as Cisco Modeling Labs and pyATS)	5.3	DevNet CML/pyATS pages	
5.4	Describe the components and benefits of CI/CD pipeline in infrastructure automation	5.4	—	
5.5	Describe the principles of infrastructure as code	5.5	Terraform lab	
5.6	Describe the capabilities of automation tools such as Ansible, Terraform, and Cisco NSO	5.6	`labs/08_ansible`, `labs/09_terraform`	
5.7	Identify the workflow being automated by a Python script that uses Cisco APIs including ACI, Meraki, Cisco Catalyst Center, and RESTCONF	5.7	`labs/05` + `restconf_get_interfaces.py`	
5.8	Interpret the workflow being automated by an Ansible playbook (management packages, user management related to services, basic service configuration, and start/stop)	5.8	`playbook.yml`	
5.9	Interpret the workflow being automated by a bash script (such as file management, app install, user management, directory navigation)	5.9	WSL	
5.10	Interpret the results of a RESTCONF or NETCONF query	5.10	sample JSON/XML + live GET	
5.11	Interpret basic YANG models	5.11	`sample.yang`	
5.12	Interpret a unified diff	5.12	`example.diff`	
5.13	Describe the principles and benefits of a code review process	5.13	GitHub PR (optional)	
5.14	Interpret a sequence diagram that includes API calls	5.14	mermaid in guide	
6.0	Network Fundamentals — **15%**	Domain 6 intro	—	

Obj	Official objective	Guide section	Lab	Status
6.1	Describe the purpose and usage of MAC addresses and VLANs	6.1	—	
6.2	Describe the purpose and usage of IP addresses, routes, subnet mask / prefix, and gateways	6.2	`prefix_to_hosts`	
6.3	Describe the function of common networking components (such as switches, routers, firewalls, and load balancers)	6.3	—	
6.4	Interpret a basic network topology diagram with elements such as switches, routers, firewalls, load balancers, and port values	6.4	mermaid in guide	
6.5	Describe the function of management, data, and control planes in a network device	6.5	—	
6.6	Describe the functionality of these IP Services: DHCP, DNS, NAT, SNMP, NTP	6.6	—	
6.7	Recognize common protocol port values (such as, SSH, Telnet, HTTP, HTTPS, and NETCONF)	6.7	Final review table	
6.8	Diagnose application connectivity issues (NAT problem, Transport Port blocked, proxy, and VPN)	6.8	`diagnose.py`	
6.9	Explain the impacts of network constraints on applications	6.9	`diagnose.py`	

Coverage verification

Every printed objective in the official three-page blueprint is listed above and has a matching `##` heading in the complete study guide.

Gaps in the *uploaded* Cisco files (not gaps in the guide)

The PDF is an outline. The Learning Matrix mostly cites the Official Cert Guide and Cisco U. Those books/courses are **not** in this folder. The study guide therefore teaches from Cisco DevNet, IOS XE programmability guides, IETF RFCs, and upstream Python/Git/Docker/Ansible/Terraform docs.

If a DevNet Always-On tile (Meraki, Catalyst Center, SD-WAN, ISE) is offline, use the URL/auth patterns in Domain 3 plus reservable sandboxes or a personal non-production API key. IOS XE Always-On / Cat8K remains the default for NETCONF/RESTCONF.